

Movie Show Investing — Replit Prompt Guide

Full-stack build for movieshowinvesting.com · React + Flask on Replit · Bunny · Mailjet · Firebase Auth · Updated September 27, 2026

Overview

This guide builds the Movie Show Investing test site in nine paste-ready prompts. The filmmaker side can launch after Prompt 6; the investor side launches after Prompt 8, two to three weeks later.

What's being built

- A stealth, two-sided marketplace owned by AYe Producer, Inc. and run outside The AYeList. Filmmakers set offers to investors; investors set minimum terms; projects that meet an investor's terms are marked as matches. Every pledge is non-binding and no money moves.
- **Filmmaker side:** a 6-screen flow that sorts projects into a slate, shows personal payback math, captures their offer to investors, and creates a private project page and share card.
- **Investor side:** a 5-page flow (amount, terms, interests, matches, about you) that ends in a signed, non-binding indication of interest and a confirmed pledge.
- **Trust:** richer project pages, verified email and phone badges, and a private "Ask the filmmaker" relay.
- **Admin:** summary, pledges by project and location, funnels, the market (offers vs minimums), price test, queues, messages, channels, and CSV export.

Why it's built this way

- **One app, one deploy:** Flask serves the built React app and the API. Same stack as The AYeList.
- **Bunny for all media:** Stream for trailers and pilots; Storage + CDN for posters and share-card images.
- **Mailjet for all email:** confirmations, sign-in links, questions and answers.
- **Firebase Auth only:** email-link sign-in (no passwords) and phone verification. Every other piece of data lives in Postgres.
- **Everything is data:** every screen saves as the person goes, so drop-off and pricing answers become test results.

How to run the prompts

- Paste one prompt at a time into Replit Agent. Each prompt ends with a hard stop.
- The agent ends each phase with "ready for testing," never "done." Verify with its raw output and the Visual Guide test scenarios before pasting the next prompt.
- Prompt 0 creates DECISIONS.md and PLAN.md. Every later prompt tells the agent to follow them.
- Keys and passwords live only in Replit Secrets, never in code.

Build order

#	Prompt	What it delivers	Gate
0	Project rules	DECISIONS.md (locked rules) and PLAN.md (phases)	Files exist
1	Setup and shell	React + Flask app, Postgres, Firebase client, layout, home, legal pages, visitor + UTM capture	Home loads
2	Data model and admin	All tables, progress saving, admin tables, CSV export	S2 passes
3	Filmmaker flow	Screens 1-6, calculator tabs, offer to investors, price test, resume	F1-F5 pass
4	Project page, result, share, showcase	Project page with offer, badges, trust fields, securities notice, share card, previews, Bunny uploads, moderation	F6-F8 pass
5	Emails, sign-in, verification, Ask the filmmaker	Mailjet emails, Firebase email-link sign-in, phone badge, private filmmaker page, message relay	F9-F11 pass
6	Hardening and deploy	Mixpanel, Clarity, Turnstile, rate limits, legal pages, domain	Launch filmmaker side
7	Explore and investor flow	Explore with matching, 5-page flow, equal split, market figures, lineup bar	I1-I6 pass
8	Pledge confirmation and lineup page	Signed indication of interest, Confirm my pledge, lineup page, Find my lineup, final QA	Launch investor side

Before you start

- ☐ **Replit:** a new Repl with Replit's built-in Postgres turned on.
- ☐ **Firebase:** a project with Authentication on; enable the **Email link (passwordless)** and **Phone** sign-in methods; add movieshowinvesting.com and your Replit dev URL as authorized domains; create a service-account key (JSON) for the server.
- ☐ **Bunny Stream:** one video library (Library ID + API key).
- ☐ **Bunny Storage:** one storage zone with a pull zone (zone name, storage password, region host, pull-zone URL).
- ☐ **Mailjet:** API key and secret; sender domain movieshowinvesting.com authenticated (SPF, DKIM, DMARC).
- ☐ **Mixpanel, Microsoft Clarity, Cloudflare Turnstile:** one project or widget each.
- ☐ **Calendly:** filmmaker chat (15 min), accredited briefing (30 min), retail group Q&A (30 min, weekly).
- ☐ **Domain:** DNS access for movieshowinvesting.com.
- ☐ **Content:** two or three real projects, with filmmaker permission, to seed Explore.

Replit Secrets

Secret name	What it holds
ADMIN_PASSWORD, FLASK_SECRET_KEY	Admin password; long random session key
FIREBASE_API_KEY, FIREBASE_AUTH_DOMAIN, FIREBASE_PROJECT_ID, FIREBASE_APP_ID	Firebase web config (used by the frontend)
FIREBASE_SERVICE_ACCOUNT	The service-account JSON, pasted whole (used by the server)
BUNNY_STREAM_LIBRARY_ID, BUNNY_STREAM_API_KEY	Bunny Stream
BUNNY_STORAGE_ZONE, BUNNY_STORAGE_PASSWORD, BUNNY_STORAGE_HOST, BUNNY_CDN_URL	Bunny Storage + CDN
MAILJET_API_KEY, MAILJET_API_SECRET, MAIL_FROM	Mailjet
TURNSTILE_SITE_KEY, TURNSTILE_SECRET_KEY	Cloudflare Turnstile
MIXPANEL_TOKEN, CLARITY_PROJECT_ID	Analytics
CALENDLY_FILMMAKER, CALENDLY_ACCREDITED, CALENDLY_RETAIL	The three Calendly links
SITE_URL	https://movieshowinvesting.com

Prompt 0 — Project rules

PASTE INTO REPLIT AGENT

You are building moviesthowinvesting.com. Before writing any app code, create two files in the repo root and nothing else.

1. DECISIONS.md with these locked rules (do not change them unless I say so):

- Brand: "Movie Show Investing", a filmmaker-investor matching platform that collects non-binding pledges. Owned by AYe Producer, Inc. The AYeList is named ONLY on the FAQ page.
- Marketplace: each project has an offer to investors, per \$100 pledged. Defaults: Distribution \$125, Production \$150, Idea \$175. Filmmakers may offer \$125, \$150, \$175, \$200 or another amount. Floor: never below \$125. "I'd need to offer less than \$125" is saved as data and the project lists at \$125.
- Platform fee per \$100, added on top of the offer: Distribution \$25 (price test group B: \$15), Production \$20, Idea \$15. Filmmaker pays back = offer + fee.
- After payback, filmmaker keeps: Distribution \$67, Production \$60 of every \$100; Idea keeps all.
- The offer is a rate: $\text{payback goal} = \text{pledge} \times \text{offer} / 100$.
- Filmmaker side may say earnings go to investors until paid back. Investor side must NEVER state who gets paid first.
- Investor wording: "Payback goal: \$150 for every \$100" (using the project's offer). Never "return", "earn", "expected", "realistic".
- Next to every investor number: "Returns aren't guaranteed. You may get back less, or nothing."
- Securities notice at the top of Explore and every project page: "Pledge your interest in future investment opportunities. If a project opens for investment, it will be offered only in compliance with securities laws, and you'll get full offering documents before you decide."
- Footer on every page: "Pledges are non-binding. No money is collected. This is not an offer to sell securities."
- Pledge limits: \$100 minimum total, \$25 minimum per project, up to 4 projects under \$150, up to 5 at \$150+. One lineup per email. Default split is equal.
- Answer ladder (per \$100): 125, 150, 175, 200, 250+, Other. Investors also have "Not interested" per slate type.
- Market figures ("Most investors here look for at least \$X", "Filmmakers typically offer \$X") show only after 20+ answers, and never on the investor terms page.
- Public pledge counts include only confirmed pledges.
- Firebase is used for Auth only (email-link sign-in, phone verification). All other data lives in Postgres.
- No payments, banking, escrow, Greenlights, leaderboards, lineup editing, public Q&A, or direct messages.
- Every count shown to users is a real number from the database.

2. PLAN.md listing Phases 1-8 exactly as I give them, each with status "not started".

Rules for every phase: build only what the phase asks. Keep secrets in Replit Secrets. When a phase is finished, stop and reply "ready for testing" with the output of `git show --stat`, a list of changed files, and how to test. Never say "done". Never start the next phase on your own.

Verify: open both files and check every number matches this guide.

Prompt 1 — Setup and shell

PASTE INTO REPLIT AGENT

Follow DECISIONS.md. Phase 1: setup and page shell only.

Build:

- A Flask backend using SQLAlchemy with Replit's Postgres via DATABASE_URL.
- A React (Vite) + Tailwind frontend in /frontend. Flask serves /frontend/dist and returns index.html for any non-API route. API routes live under /api.
- One run command that builds the frontend and starts Flask.
- Firebase: initialize the Firebase web SDK in the frontend (firebase/app, firebase/auth) from the FIREBASE_* secrets exposed through GET /api/config. Initialize the Firebase Admin SDK in Flask from FIREBASE_SERVICE_ACCOUNT. No sign-in screens yet.
- Mobile-first layout: header ("Movie Show Investing", "Find my lineup" link placeholder), content max 480px on phones and 720px on desktop, footer with the DECISIONS.md footer text and links to /privacy, /terms, /disclaimers, /faq.
- Home page: headline "Fund your next film with investors, not T-shirts.", buttons "I'm a filmmaker" (/start/filmmaker) and "I want to invest" (/invest), and "X filmmakers have joined" from GET /api/stats (hidden if zero).
- Placeholder pages /privacy, /terms, /disclaimers. /faq with these five questions and answers exactly as in PLAN.md (I will paste them).
- Visitor tracking: random visitor_id cookie (1 year); capture utm_source, utm_medium, utm_campaign and ?ref= codes.

Stop at "ready for testing".

Code helper — Flask entry point with Firebase Admin

```
# app.py (shape of the Flask entry point)
import os, json
from flask import Flask, send_from_directory
from flask_sqlalchemy import SQLAlchemy
import firebase_admin
from firebase_admin import credentials

DIST = os.path.join(os.path.dirname(__file__), "frontend", "dist")
app = Flask(__name__, static_folder=DIST, static_url_path="")
app.config["SQLALCHEMY_DATABASE_URI"] = os.environ["DATABASE_URL"]
app.secret_key = os.environ["FLASK_SECRET_KEY"]
db = SQLAlchemy(app)
firebase_admin.initialize_app(
    credentials.Certificate(json.loads(os.environ["FIREBASE_SERVICE_ACCOUNT"])))

@app.route("/", defaults={"path": ""})
@app.route("/<path:path>")
def spa(path):
    full = os.path.join(DIST, path)
    if path and os.path.exists(full):
        return send_from_directory(DIST, path)
    return send_from_directory(DIST, "index.html")
```

FAQ text to paste into PLAN.md

1. **What is Movie Show Investing?** A platform where filmmakers share projects and investors pledge interest, so both sides can find each other before investment opens.

2. **Is my pledge an investment?** No. No trades happen here and no money is collected. A pledge is a non-binding indication of interest. If a project opens for investment, you'll be invited to decide then.
3. **How will money flow when investment opens?** Viewers rent, buy, or license the film → our affiliated distribution platform collects the revenue (payment processing fees come off first) → automated revenue splits pay investors, the filmmaker, and the platform according to the project's terms.
4. **Who's the distribution platform?** Movie Show Investing and The AYeList are both run by AYe Producer, Inc. The AYeList is our distribution channel, with on-demand streaming and automated revenue splits.
5. **How are my ideas protected?** Share loglines and synopses, not full scripts. Consider registering your script with the U.S. Copyright Office or the WGA registry before sharing widely.

Prompt 2 — Data model and admin

PASTE INTO REPLIT AGENT

Follow DECISIONS.md. Phase 2: tables, progress saving, and admin. No user-facing flows yet.

SQLAlchemy models (created_at on all):

- Visitor: visitor_id, utm_source, utm_medium, utm_campaign, ref_code_used, price_group ("A"/"B", assigned once, 50/50).
- Filmmaker: name, email, phone, city, state, favorite_genres (list), chat_opt_in, no_project_yet, visitor_id, own_ref_code, firebase_uid, email_verified, phone_verified, funding_sources (list), funding_other, reached_goal, funding_experience.
- Project: filmmaker_id, slug, title, format, genre, genre_other, stage ("distribution"/"production"/"idea"/"other"), stage_other, logline, trailer_url, pilot_url, bunny_video_id, poster_url, share_image_url, budget, budget_from_example, price_group, deal_answer ("yes"/"maybe"/"no"), offer_per100 (integer, >= 125), offer_other_text, wants_lower (bool), payback_terms ("works"/"need_some"/"other"), payback_terms_other, synopsis, team_links (list), money_use, distribution_plan, showcase_requested, approved (default false), hidden (default false).
- Investor: name, email (unique), phone, city, state, zip, amount_choice, accredited, experience (list), experience_other, motivations (list), motivations_other, favorite_genres (list), stages (list), feel_safe_text, call_opt_in, path ("build"/"auto"/"just"), preference, preference_why, signature_name, signed_at, confirmed_at, firebase_uid, visitor_id, own_ref_code.
- Pledge: investor_id, project_id (nullable = unallocated), amount, confirmed (bool, set when the investor confirms).
- InvestorMinimum: investor_id, slate, min_per100 (nullable), other_text, not_interested (bool).
- Message: project_id, investor_name, investor_email, question, answer, answer_token_hash, asked_at, answered_at, reported (bool), hidden (bool).
- FlowProgress: session_id, flow, last_screen, answers (JSON), completed, updated_at.
- EmailLog: to, type, status.

API: POST /api/progress (upsert by session_id); GET /api/price-group (assign once).

Admin at /admin (ADMIN_PASSWORD, session login), each section a table with a CSV export:

1. Summary: filmmakers, projects, showcased; investors, confirmed pledges; total pledged (to projects vs unallocated); average pledge; accredited count and dollars.
2. Pledges by project: title, slate, offer_per100, confirmed pledgers, total, accredited pledgers, top city.
3. By location: state, city, investors, pledged total, filmmakers.
4. Funnels: filmmaker screens 1-6 and investor pages 1-5: reached, drop-off %.
5. Market: per slate, count of filmmaker offers at each ladder step, wants_lower count, count of investor minimums at each step, not-interested count.
6. Price test: Distribution group A vs B: started, completed, deal_answer split, offers chosen.
7. Queues: approvals (Approve / Hide), calls (accredited first, then retail, then filmmaker chats), unanswered questions.
8. Messages: project, investor, question, answer, asked, response time, reported; Hide button.
9. Channels: sign-ups by utm_source and by ref code.
10. Email log.

Stop at "ready for testing".

Code helper — pricing with per-project offers

```
# pricing.py — single source of truth (whole dollars, no floats)
SLATES = {
    "distribution": {"default": 125, "fee": 25, "fee_b": 15, "keep": 67},
    "production": {"default": 150, "fee": 20, "fee_b": 20, "keep": 60},
    "idea": {"default": 175, "fee": 15, "fee_b": 15, "keep": 100},
}
FLOOR = 125

def deal(slate, budget, offer=None, group="A"):
    s = SLATES[slate]
    offer = max(FLOOR, offer or s["default"])
    fee = s["fee_b"] if (group == "B" and slate == "distribution") else s["fee"]
    investors = budget * offer // 100
    platform = budget * fee // 100
    return {"offer": offer, "investors": investors, "fee": platform,
            "payback": investors + platform, "per100": offer + fee, "keep": s["keep"]}

def payback_goal(pledge, offer):
    return pledge * offer // 100
```

Code helper — market figures

```
# market figure: median, only with enough answers
def market_figure(values, min_n=20):
    vals = sorted(v for v in values if v)
    if len(vals) < min_n:
        return None
    return vals[len(vals) // 2]
```

Verify: scenario S2 in the Visual Guide.

Prompt 3 — Filmmaker flow

PASTE INTO REPLIT AGENT

Follow DECISIONS.md. Phase 3: the filmmaker flow at /start/filmmaker. Six screens, progress bar, back button, big tap targets, auto-advance on single-choice taps. Save to POST /api/progress after every screen; session_id in localStorage so the same device resumes.

Screen 1 "Where's your project at?": Distribution phase (looking for distribution, or already distributed) / Short or pilot I want to develop / Script or idea / Other (text). Sets slate; Other uses idea math.

Screen 2 "Your project": title, Movie/Show toggle, genre chips (Horror, Drama, Comedy, Thriller, Documentary, Sci-Fi, Other + text), logline, trailer link (optional), short/pilot link (optional, only for short or pilot). Checkbox "I don't have a project yet. I want to participate in the future." skips to Screen 6.

Screen 3 "Your deal": tabs "Example budget" / "Your budget", shared with Screen 4.

- Example chips: Distribution \$25K, \$50K, \$100K. Production movie \$200K; show \$350K, \$450K. Idea movie \$50K; show \$65K.
- Your budget: \$ input with commas, pre-filled with the last example; required; any amount accepted.
- Receipt at the slate's default offer: YOU RAISE / Investors get back / Platform fee / YOU PAY BACK (with per-\$100 lines) and "After that, you keep \$X of every \$100 your film earns" (Idea: "you keep it all"). Distribution uses the visitor's price group.
- "Would you take this deal?": Yes, I'd take it / Maybe, if I could negotiate / No, not at these terms. Save as deal_answer.

Screen 4 "Your offer to investors": same tabs and budget.

- "Standard for [Slate]: \$X per \$100". Offer chips 125, 150, 175, 200, Other (\$ input, must be ≥ 125 , friendly message if lower). If deal_answer is yes, preselect the default; otherwise nothing is preselected.
- Checkbox "I'd need to offer less than \$125" (sets wants_lower; offer saved as 125).
- Live receipt at the chosen offer: Investors get back / Platform fee / YOU PAY BACK.
- Two-phase card: "PHASE 1 Until \$[payback] is paid back, earnings go to investors" and "PHASE 2 After that: YOU KEEP \$[keep] OF EVERY \$100" (largest text on the card).
- Market line from GET /api/market/[slate]: "Most investors here look for at least \$X per \$100." Hidden until 20+ investor answers.
- "How payback works": "If your film earns \$10,000 a month, all of it goes to investors until \$[payback] is paid back, about [N] months. After that, you keep \$[keep*100] of every \$10,000." Options: Works for me / I need some from day one / Other (text).
- Small note: "Your offer appears on your project page."

Screen 5 "How did you finance your last project?": multi-select (Own money, Friends & family, Kickstarter / Indiegogo / Seed&Spark, Grants, Investors, Studios, Haven't yet, Other + text); then "Did you reach your goal?" Yes/No and "What was your experience?"

Screen 6 "Almost done": name, email, phone (optional), city, state, favorite genres, checkbox "Open to a quick 15-minute chat". Submit creates Filmmaker + Project and routes to /start/filmmaker/done (placeholder for Phase 4).

Stop at "ready for testing".

Code helper — live calculator (frontend)

```
// frontend/src/lib/pricing.js — mirror of pricing.py
export const FLOOR = 125;
export const SLATES = {
  distribution: { def: 125, fee: 25, feeB: 15, keep: 67 },
  production:   { def: 150, fee: 20, feeB: 20, keep: 60 },
  idea:         { def: 175, fee: 15, feeB: 15, keep: 100 },
};
export function deal(slate, budget, offer, group = "A") {
  const s = SLATES[slate];
  const o = Math.max(FLOOR, offer || s.def);
  const fee = group === "B" && slate === "distribution" ? s.feeB : s.fee;
  const investors = Math.floor((budget * o) / 100);
  const platform = Math.floor((budget * fee) / 100);
  return { offer: o, investors, platform, payback: investors + platform,
    per100: o + fee, keep: s.keep };
}
export const money = (n) => "$" + Math.round(n).toLocaleString("en-US");
export const LADDER = [125, 150, 175, 200];
```

Code helper — payback example line

```
// Screen 4 payback example
const months = Math.round(d.payback / 10000);
const keepLine = d.keep === 100 ? "you keep all of it"
  : `you keep ${money(d.keep * 100)} of every $10,000`;
```

Verify: scenarios F1-F5.

Prompt 4 — Project page, result, share card, showcase

PASTE INTO REPLIT AGENT

Follow DECISIONS.md. Phase 4.

1. Result /start/filmmaker/done: "[TITLE] fits the [Slate] Slate" with their numbers at their offer (raise, pays back, keeps after). Short CSS celebration. Then the share card, then the showcase screen.

2. Share card: 1200x630 PNG made with Pillow: title, genre, slate, and "I want investors, not crowdfunding rewards." No dollar amounts. Upload to Bunny Storage at share/[slug].png. Buttons: Instagram (download), LinkedIn, X, Copy link. Link: SITE_URL/p/[slug]?ref=[filmmaker own_ref_code].

3. Project page /p/[slug] (unlisted by default):

- The securities notice from DECISIONS.md at the top.
- Video (Bunny embed if bunny_video_id, else the pasted link, else plain link), poster or auto thumbnail, title, logline, genre, slate label (Finished film / Short or pilot to develop / Script or idea), movie/show.
- Badges "Email verified" and "Phone verified" when true.
- "Raising: \$X", "Payback goal: \$[offer_per100] for every \$100", the disclaimer line, and "\$X pledged by N people" from CONFIRMED pledges only (hidden if zero).
- Strengthen fields when present: Synopsis, Team (links), What the money pays for, Distribution plan.
- Full disclaimer; "How the money flows" block; "Where the money comes from" list (Streaming rentals and purchases: Live; Outside distributors and streamers: In progress; Screenings, events, and merch: In progress). No payment order. Never name The AYeList.
- Buttons "I'm interested" and "Ask the filmmaker" (placeholders until Phases 5 and 7).

4. Link previews: Flask renders /p/[slug] with Open Graph and Twitter tags injected into index.html. Unapproved pages get noindex.

5. Showcase screen: "Put [TITLE] in front of investors." Yes, showcase it / Keep it private for now. Below: "Strengthen your page (optional)": synopsis (up to 800 characters), team links (up to 5 URLs), what the money pays for, distribution plan, trailer upload (Bunny Stream, direct TUS upload, 500MB max), poster upload (Bunny Storage, 5MB, jpg/png/webp). Add a note: "Share a synopsis, not your full script."

6. Explore (Phase 7) lists only approved, not hidden projects. Admin Approve/Hide controls this.

7. Auto thumbnails from YouTube or Vimeo oEmbed when no poster.

Stop at "ready for testing".

Code helper — share previews

```
# Link preview for a single-page app
from markupsafe import escape
@app.route("/p/<slug>")
def project_page(slug):
    p = Project.query.filter_by(slug=slug, hidden=False).first_or_404()
    html = open(os.path.join(DIST, "index.html")).read()
    tags = (
        f'<meta property="og:title" content="{escape(p.title)} | Movie Show Investing">'
        f'<meta property="og:description" content="{escape(p.logline or "")}">'
        f'<meta property="og:image" content="{escape(p.share_image_url or "")}">'
        f'<meta name="twitter:card" content="summary_large_image">'
    )
    if not p.approved:
        tags += '<meta name="robots" content="noindex">'
    return html.replace("<!--OG-->", tags) # <!--OG--> sits inside <head>
```

Code helper — Bunny Storage

```
# Bunny Storage upload (posters, share cards)
import requests
def bunny_put(path, data, content_type):
    host, zone = os.environ["BUNNY_STORAGE_HOST"], os.environ["BUNNY_STORAGE_ZONE"]
    r = requests.put(f"https://{host}/{zone}/{path}", data=data,
                    headers={"AccessKey": os.environ["BUNNY_STORAGE_PASSWORD"],
                             "Content-Type": content_type})
    r.raise_for_status()
    return f"{os.environ['BUNNY_CDN_URL']}/{path}"
```

Code helper — Bunny Stream direct upload

```
# Bunny Stream: create the video, then sign a direct browser (TUS) upload
import hashlib, time
LIB, KEY = os.environ["BUNNY_STREAM_LIBRARY_ID"], os.environ["BUNNY_STREAM_API_KEY"]

def start_video_upload(title):
    r = requests.post(f"https://video.bunnycdn.com/library/{LIB}/videos",
                     json={"title": title}, headers={"AccessKey": KEY})
    r.raise_for_status()
    video_id = r.json()["guid"]
    expires = int(time.time()) + 3600
    sig = hashlib.sha256(f"{LIB}{KEY}{expires}{video_id}".encode()).hexdigest()
    # browser uploads to https://video.bunnycdn.com/tusupload with headers:
    # AuthorizationSignature=sig, AuthorizationExpire=expires, VideoId, LibraryId
    return {"video_id": video_id, "signature": sig, "expires": expires, "library_id": LIB}
# player: https://iframe.mediadelivery.net/embed/{LIB}/{video_id}
```

Code helper — filmmaker share card

```
# Share card image (Pillow), 1200x630, no dollar amounts
from PIL import Image, ImageDraw, ImageFont
def share_card(title, genre, slate):
    img = Image.new("RGB", (1200, 630), "#14151A")
    d = ImageDraw.Draw(img)
    big = ImageFont.truetype("DejaVuSans-Bold.ttf", 64)
    mid = ImageFont.truetype("DejaVuSans.ttf", 34)
    d.text((80, 90), title.upper()[:28], font=big, fill="#FFFFFF")
    d.text((80, 190), f"{genre} · {slate} Slate", font=mid, fill="#B9B3F5")
    d.text((80, 380), "I want investors,", font=big, fill="#F2B84B")
    d.text((80, 460), "not crowdfunding rewards.", font=big, fill="#F2B84B")
    return img
```

Verify: scenarios F6–F8. Paste a /p/ link into LinkedIn Post Inspector to confirm the preview.

Prompt 5 — Emails, sign-in, verification, Ask the filmmaker

PASTE INTO REPLIT AGENT

Follow DECISIONS.md. Phase 5.

1. Mailjet: send all email through the Mailjet v3.1 API (MAILJET_API_KEY, MAILJET_API_SECRET, MAIL_FROM). Log every send in EmailLog.
2. Firebase email-link sign-in (no passwords):
 - Server creates sign-in links with the Firebase Admin SDK (generate_sign_in_with_email_link) pointing to SITE_URL/signin/finish?next=[path], and sends them through Mailjet (branded emails, not Firebase's default sender).
 - /signin/finish completes sign-in with signInWithEmailLink (ask for the email if it isn't stored on this device), then goes to next.
 - API routes for private pages require a Firebase ID token (Authorization: Bearer). The server verifies it and matches the email to the record.
 - First successful sign-in by a filmmaker sets email_verified = true.
3. Filmmaker confirmation email: subject "Your project is in: [TITLE]". Slate, offer, raise and payback numbers, share link, button "View my project" (sign-in link to /me). Add "Book your 15-minute chat" with CALENDLY_FILMMAKER if chat_opt_in. "No project yet" sign-ups get a short "You're on the list" email.
4. Private filmmaker page /me (signed-in only): project, numbers, share card, status (Private / Waiting for review / Showcased), referral count, "Verify your phone" button, and "Questions from investors" (unanswered first, with an Answer box).
5. Phone verification: on /me, "Verify your phone" links a phone number to the signed-in Firebase user with an SMS code (linkWithPhoneNumber + invisible reCAPTCHA). On success the server sets phone_verified = true.
6. Ask the filmmaker (private relay):
 - Project page button opens a form: name, email, question (500 characters max), Turnstile.
 - Save a Message with a hashed answer token. Email the filmmaker: the question, the asker's first name only, and a button to /answer/[token] (also answerable from /me).
 - /answer/[token] shows the question and an answer box. On submit, save the answer and email it to the asker with the project link. Emails are never shared between the two.
 - Every email includes "Report this message" (sets reported). Limit: 5 questions per email per day; 20 per IP per day.
 - The form states: "Messages are stored and may be reviewed by Movie Show Investing for safety."

Stop at "ready for testing".

Code helper — Mailjet

```
# Mailjet send (pip install mailjet_rest)
from mailjet_rest import Client
mj = Client(auth=(os.environ["MAILJET_API_KEY"], os.environ["MAILJET_API_SECRET"]), version="v3.1")

def send_email(to_email, to_name, subject, html, kind):
    data = {"Messages": [{
        "From": {"Email": os.environ["MAIL_FROM"], "Name": "Movie Show Investing"},
        "To": [{"Email": to_email, "Name": to_name}],
        "Subject": subject, "HTMLPart": html,
    }]}
    r = mj.send.create(data=data)
    db.session.add(EmailLog(to=to_email, type=kind, status=r.status_code))
    db.session.commit()
    return r.status_code
```

Code helper — Firebase Admin (server)

```
# Firebase: branded sign-in links + verifying the signed-in user
from firebase_admin import auth
from flask import request

def signin_link(email, next_path):
    settings = auth.ActionCodeSettings(
        url=f"{os.environ['SITE_URL']}/signin/finish?next={next_path}",
        handle_code_in_app=True)
    return auth.generate_sign_in_with_email_link(email, settings)

def signed_in_email():
    h = request.headers.get("Authorization", "")
    if not h.startswith("Bearer "):
        return None
    try:
        return auth.verify_id_token(h[7:]).get("email", "").lower()
    except Exception:
        return None
```

Code helper — Firebase (frontend)

```
// frontend: finish email-link sign-in, then phone verification
import { getAuth, isSignInWithEmailLink, signInWithEmailLink,
    RecaptchaVerifier, linkWithPhoneNumber } from "firebase/auth";
const auth = getAuth();

export async function finishSignIn() {
    if (!isSignInWithEmailLink(auth, window.location.href)) return;
    const email = localStorage.getItem("msi_email") || window.prompt("Confirm your email");
    await signInWithEmailLink(auth, email, window.location.href);
}

export async function sendPhoneCode(phoneE164) {
    const verifier = new RecaptchaVerifier(auth, "phone-captcha", { size: "invisible" });
    return linkWithPhoneNumber(auth.currentUser, phoneE164, verifier); // returns confirmer
}
// later: await confirmer.confirm(code); then POST /api/verify-phone with the ID token
```

Code helper — Ask the filmmaker relay

```
# Ask the filmmaker: store the question, email a private answer link
import secrets, hashlib
@app.post("/api/projects/<slug>/ask")
@limiter.limit("5/day", key_func=lambda: (request.get_json() or {}).get("email", "").lower())
def ask(slug):
    b = request.get_json() or {}
    if not turnstile_ok(b.get("turnstile"), request.remote_addr):
        return {"message": "Please try again."}, 400
    p = Project.query.filter_by(slug=slug, hidden=False).first_or_404()
    token = secrets.token_urlsafe(32)
    m = Message(project_id=p.id, investor_name=b["name"], investor_email=b["email"].lower(),
               question=b["question"][:500],
               answer_token_hash=hashlib.sha256(token.encode()).hexdigest())
    db.session.add(m); db.session.commit()
    send_question_email(p, m, f"{os.environ['SITE_URL']}/answer/{token}")
    return {"message": "Sent. You'll get the answer by email."}
```

Verify: scenarios F9-F11. Check emails land in the inbox, not spam.

Prompt 6 — Hardening and deploy (filmmaker launch)

PASTE INTO REPLIT AGENT

Follow DECISIONS.md. Phase 6.

- Mixpanel: fm_screen_view {screen}, fm_complete {slate, offer, deal_answer, price_group}, fm_share_click {channel}, fm_showcase {choice}, ask_sent, ask_answered, plus utm_source. Microsoft Clarity on all pages.
- Cloudflare Turnstile on the filmmaker final submit and the Ask form (server verifies). Hidden honeypot field on both.
- Flask-Limiter: 10 submits per IP per hour; 60 progress saves per minute.
- Legal pages: placeholder text for /privacy, /terms, /disclaimers. Privacy names AYe Producer, Inc., says messages may be reviewed for safety, and gives a delete-my-data email address.
- Deploy with Replit Deployments; connect moviesshowinvesting.com (show me the DNS records). Add the domain to Firebase authorized domains.

Stop at "ready for testing".

Code helper — Turnstile

```
# Turnstile server check
def turnstile_ok(token, ip):
    r = requests.post("https://challenges.cloudflare.com/turnstile/v0/siteverify",
                     data={"secret": os.environ["TURNSTILE_SECRET_KEY"],
                           "response": token, "remoteip": ip}, timeout=5)
    return r.ok and r.json().get("success") is True
```

Verify: F1-F11 on the live domain from a phone. Then launch the filmmaker side.

Prompt 7 — Explore and investor flow

PASTE INTO REPLIT AGENT

Follow DECISIONS.md. Phase 7: Explore, matching, and the 5-page investor flow. Investor copy follows the wording rules exactly.

Explore /explore: securities notice at top. Header "Build your lineup of movies and shows." Filters: slate type, genre, movie/show, raise size, payback goal; sort by newest or payback goal. Buttons: "Auto-build my lineup", "Just pledge an amount". Cards (approved, not hidden): poster, title, genre, movie/show, stage label, verified badges, logline, "Raising: \$X", "Payback goal: \$[offer] for every \$100", disclaimer line, confirmed pledge count (hidden if zero), "Matches your terms" badge when applicable, "Add to my lineup" (starts the flow at Page 1 with this project in the lineup). "I'm interested" on the project page does the same.

Investor flow /invest (save progress after every page, flow "investor"):

Page 1 "Your amount": chips \$100, \$500, \$1,000, \$10,000, Other (\$100 minimum).

Page 2 "Your terms": "The least you'd need back for every \$100 you put in", one row each for Finished film, Short or pilot to develop, Script or idea: 125, 150, 175, 200, 250+, Other, Not interested. Then lender / investor / either / not sure + optional Why. Nothing preselected. Show NO offers or market figures on this page.

Page 3 "Your interests": genre chips, stage chips, motivations (Returns, Supporting indie filmmakers, Spreading risk / alternative investments, Perks like premieres or credits, Other), "What would make you feel safe investing in movies and shows?".

Page 4 "Your matches": approved projects filtered to their interests; "Matches your terms" badge where the offer meets their minimum for that slate type (not interested = never a match); matches sorted first. Show "Filmmakers typically offer \$X" per slate (hidden under 20 offers). They tap projects to add; the amount splits equally (whole dollars, remainder to the first); each row editable with chips and custom; limits enforced with friendly messages. Options on this page: "Auto-build for me" (matching projects across slate types, then genres) and "Just pledge my amount" (one unallocated pledge).

Lineup bar (sticky from Page 4): "MY LINEUP: \$X across N projects", mix by slate, spread rating, "Payback goal if every project pays back in full: \$X" using each project's offer, disclaimer line.

Page 5 "About you": name, email, phone (optional), city, state, zip, accredited Yes/No/Not sure with the plain definition, experience, "Open to a call", typed full name as a signature under "I'm signing a non-binding indication of interest.", required acknowledgment checkbox, Turnstile. Save signed_at. If the email already has a lineup, do not create another; show "You already have a lineup. We've sent you the link." (email in Phase 8).

On submit create Investor, Pledges (confirmed = false), InvestorMinimums, then go to /lineup/done (placeholder for Phase 8).

Stop at "ready for testing".

Code helper — lineup rules and matching

```
// frontend/src/lib/lineup.js — limits, split, spread, match
export function lineupErrors(rows) {
  const total = rows.reduce((a, r) => a + r.amount, 0);
  const errs = [];
  if (rows.some((r) => r.amount < 25)) errs.push("Each project needs at least $25.");
  if (total < 100) errs.push("Your lineup needs at least $100 total.");
  const max = total < 150 ? 4 : 5;
  if (rows.length > max) errs.push(total < 150
    ? "Pledges under $150 can include up to 4 projects."
    : "A lineup can include up to 5 projects.");
  return errs;
}
export function equalSplit(total, n) {
  const each = Math.floor(total / n);
  return Array.from({ length: n }, (_, i) => each + (i === 0 ? total - each * n : 0));
}
export function spreadRating(rows) {
  const n = rows.length, slates = new Set(rows.map((r) => r.slate)).size;
  if (n <= 1) return { dots: 1, label: "All in one" };
  if (n >= 4 && slates === 3) return { dots: 4, label: "Fully spread" };
  if (n >= 4 && slates >= 2) return { dots: 3, label: "Well spread" };
  return { dots: 2, label: "A little spread" };
}
// minimums: { distribution: 150 | null | "no", production: ..., idea: ... }
export function matchesTerms(project, minimums) {
  const min = minimums[project.slate];
  if (min === "no") return false;
  return min == null || project.offer >= min;
}
export const paybackGoal = (amount, offer) => Math.floor((amount * offer) / 100);
```

Code helper — auto-build

```
// auto-build: matching projects, round-robin across slates, then new genres
export function autoBuild(total, projects, minimums) {
  const pool = projects.filter((p) => matchesTerms(p, minimums));
  const max = Math.min(total < 150 ? 4 : 5, Math.floor(total / 25));
  const bySlate = ["distribution", "production", "idea"].map((s) =>
    pool.filter((p) => p.slate === s));
  const picked = [], genres = new Set();
  while (picked.length < max && bySlate.some((l) => l.length)) {
    for (const list of bySlate) {
      if (!list.length || picked.length >= max) continue;
      const i = Math.max(0, list.findIndex((p) => !genres.has(p.genre)));
      const [p] = list.splice(i, 1);
      picked.push(p); genres.add(p.genre);
    }
  }
  const amounts = equalSplit(total, picked.length || 1);
  return picked.map((p, i) => ({ ...p, amount: amounts[i] }));
}
```

Verify: scenarios I1-I6.

Prompt 8 — Pledge confirmation and lineup page

PASTE INTO REPLIT AGENT

Follow DECISIONS.md. Phase 8: investor result, confirmation, lineup page, Find my lineup, final QA.

- Result /lineup/done: "Your lineup is in", each project with amount and "Payback goal: \$X back on your \$Y", disclaimer line, "Check your email to confirm your pledge.", the share card, and "N people joined through your link" (hidden if zero).
- Investor share card: "I've pledged to invest in [TITLE] when it opens. A pledge from you would go a long way." with the poster; link /p/[slug]?ref=[investor own_ref_code]. Toggle "Show my pledge amount" OFF by default. Instagram (download), LinkedIn, X, Copy link. Unallocated-only investors share /explore?ref=[code].
- Pledge confirmation email (Mailjet), subject "Confirm your pledge (non-binding)": lineup summary with payback goals and the disclaimer line, a copy of the signed indication of interest (typed name, date and time, acknowledgment text), the securities notice, the footer text, and a button "Confirm my pledge" = Firebase sign-in link with next=/lineup/confirm. Add CALENDLY_ACCREDITED ("Book your 30-minute briefing") for accredited "yes" with call opt-in; CALENDLY_RETAIL ("Join the weekly group Q&A") for other call opt-ins.
- /lineup/confirm (signed in): set confirmed = true on the investor's pledges and confirmed_at; show "Your pledge is confirmed." and link to /lineup.
- Private lineup page /lineup (signed in): lineup bar numbers, each pledge fixed, each project's live confirmed total, live referral count, buttons "Share my card" and "Explore more projects". Never show email, phone, city, state, zip or accredited status. View-only.
- Find my lineup: header form (email + Turnstile). Always reply "If that email has a lineup, we've sent a link to it." If the investor exists, email a fresh sign-in link with next=/lineup. Limit 3 per email per hour and 10 per IP per hour.
- Existing email in the flow: send the Find my lineup email and show "You already have a lineup. We've sent you the link."
- Mixpanel: inv_page_view {page}, inv_complete {path, total, accredited}, inv_confirmed, lineup_add, inv_share_click.
- Final pass: list any investor page missing the securities notice, a disclaimer line next to a number, or the footer.

Stop at "ready for testing".

Code helper — Find my lineup

```
# Find my lineup: same answer every time; link goes only to the real inbox
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address
limiter = Limiter(get_remote_address, app=app)

def email_key():
    return (request.get_json(silent=True) or {}).get("email", "").strip().lower()

@app.post("/api/lineup/resend")
@limiter.limit("10/hour")
@limiter.limit("3/hour", key_func=email_key)
def resend():
    body = request.get_json() or {}
    if not turnstile_ok(body.get("turnstile"), request.remote_addr):
        return {"message": "Please try again."}, 400
    inv = Investor.query.filter_by(email=email_key()).first()
    if inv:
        send_lineup_link(inv, signin_link(inv.email, "/lineup"))
    return {"message": "If that email has a lineup, we've sent a link to it."}
```

Code helper — confirm pledge

```
# Confirm my pledge: signed-in investor confirms their own pledges
from datetime import datetime, timezone
@app.post("/api/lineup/confirm")
def confirm():
    email = signed_in_email()
    inv = email and Investor.query.filter_by(email=email).first()
    if not inv:
        return {"message": "Please sign in again."}, 401
    for pl in Pledge.query.filter_by(investor_id=inv.id):
        pl.confirmed = True
    inv.confirmed_at = inv.confirmed_at or datetime.now(timezone.utc)
    db.session.commit()
    return {"message": "Your pledge is confirmed."}
```

Verify: scenarios I7-I12, then launch the investor side.

Implementation summary

Prompt	Main pages / endpoints	Services	Tests (Visual Guide)	Done when
0 Rules	DECISIONS.md, PLAN.md	—	—	Numbers match this guide
1 Setup	/, /faq, legal pages, /api/stats, /api/visit, /api/config	Postgres, Firebase init	S1	Home loads on phone; UTM saved
2 Data + admin	All models, /api/progress, /api/price-group, /admin	—	S2	10 admin tables + CSVs work
3 Filmmaker flow	/start/filmmaker (Screens 1-6), /api/market/[slate]	—	F1-F5	Math, offer floor, resume correct
4 Project + share	/start/filmmaker/done, /p/[slug], showcase, uploads	Bunny Stream, Bunny Storage	F6-F8	Preview shows; approval gates Explore
5 Emails + trust	/signin/finish, /me, /answer/[token], ask API	Mailjet, Firebase Auth	F9-F11	Sign-in, badges, relay work
6 Harden + deploy	Analytics, Turnstile, limits, domain	Mixpanel, Clarity, Turnstile	F1-F11 live	Filmmaker launch
7 Investor flow	/explore, /invest (Pages 1-5), lineup bar	—	I1-I6	Matching, split, limits correct
8 Confirm + lineup	/lineup/done, /lineup/confirm, /lineup, resend API	Mailjet, Firebase Auth, Turnstile	I7-I12	Investor launch

Keep out of scope

- Payments, banking, escrow, Greenlights, leaderboards, lineup editing, public Q&A, direct messages, tiered rates.
- If the agent suggests anything not in the phase prompt, the answer is "not in this phase."

Before each launch

- ☐ Securities attorney has answered: (1) pledges for multiple filmmakers' future offerings, (2) required notice wording, (3) matching and per-project offers.
- ☐ Mailjet SPF and DKIM verified; Firebase authorized domains include movieshowinvesting.com.
- ☐ Two or three approved projects are on Explore before the investor side opens.
- ☐ Pass line for each test is written down before marketing starts.